



RV College of Engineering®

Mysore Road, RV Vidyaniketan Post, Bengaluru - 560059, Karnataka, India

Diabetes Risk Prediction

MLOPs Tutorial Report

submitted by

Shreyas Bharadwaj 1RV23AI096

Shubhaditya Basudeo Bechan 1RV23AI099

Shreya Mohan 1RV23AI095

Shaurya Singh 1RV23AI092

Artificial Intelligence and Machine Learning

2024-2025

TABLE OF CONTENTS	Page
Phase 1: Model Development and Foundations	
Chapter 1:	
1.1 Problem Statement and Objectives	5
1.2 Project Scope and Evaluation Metrics	6
1.3 Tools and Environmental Setup	7
1.4 Literature Review	7
Chapter 2	
2.1 Understanding MLOps, Risks and Requirement Analysis	8
2.2 Risk Matrix	9
Chapter 3	
3.1 Data Collection and Exploration Techniques	10
3.2 Data profiling, cleaning and preprocessing	11
3.3 Feature Extraction and Feature Selection	12
3.4 Model Design Approaches	15
3.5 Governance and Data Version Control	15
Chapter 4	
4.1 Model Building	16
4.2 Training, Validation and Testing	16

4.3 Evaluation Metrics and Visualization	17
4.4 Experiment Tracking	20
4.5 Interpreting Model Results and Error Analysis	21
Phase 2: Deployment, Monitoring and Governance	
Chapter 5	
5.1 Revisiting Model Performance	22
5.2 Hyperparameter Tuning and Performance	22
5.3 Interpretability	23
5.4 Model Versioning and Registry	23
5.5 Documentation of Model Updates	23
Chapter 6	
6.1 Overview of CI/CD for ML	24
6.2 Tools Setup: GitHub Actions/Jenkins/GitLab CI	24
6.3 Automating Model Training and Testing	24
6.4 Building and Running a Deployment Pipeline	25
Chapter 7	
7.1 Model Monitoring Concepts and KPI's	26
7.2 Tools: Prometheus, Grafana, EvidentlyAI	26

7.3 Detecting Concept Drift and Data Drift	27
7.4 Model Retraining and Redeployment	27
7.5 Logging, Alerting and Dashboard Setup	28
Chapter 8	
8.1 Post-Deployment Performance Review	29
8.2 Continuous Improvement via Feedback Loops	29
8.3 Governance and Ethical AI Practices	29
8.4 Documentation: Model Cards, Data Cards	29
8.5 Audit and Review Checklist	30
Appendix	
A.1 List of MLOPs Tools, Features, Pros and Cons	30
A.2 Tool Installation Guides	31
A.3 Dataset Referencing and Licensing	31
A.4 Folder Structure	32
A.5 References	36
A.6 Further Reading	37

Phase 1: Model Development and Foundations

Chapter 1:

This chapter provides an introduction to the early detection of diabetes using machine learning techniques. It outlines the problem statement and objectives of the proposed system, followed by the project scope and evaluation metrics. The chapter also describes the tools and environment used for implementation. Finally, it reviews existing literature to highlight research gaps and motivate the proposed MLOps-based solution.

1.1 Problem statement and Objectives

Problem Statement

Diabetes mellitus is a chronic metabolic disorder that often remains undiagnosed in its early stages, leading to severe complications such as cardiovascular disease, kidney failure, and vision loss. Traditional diagnosis relies on periodic clinical tests and physician assessment, which may delay detection—especially in resource-constrained settings. With the growing availability of electronic health records and clinical datasets, there is a need for an **automated, data-driven system** that can predict the likelihood of diabetes at an early stage while ensuring reliability, scalability, and continuous monitoring in real-world deployment.

Objectives

The objectives of this project are:

- To develop a **machine learning–based classification system** for early diabetes detection using clinical health parameters.
- To compare multiple ML models and select the best-performing model for production deployment.
- To implement a **full MLOps pipeline** that supports training, inference, monitoring, and retraining.
- To detect **data drift and distribution shifts** in incoming patient data to maintain model reliability.
- To provide an interactive user interface for prediction, training, and monitoring.

1.2 Project Scope and Evaluation Metrics

Project Scope

The scope of this project is to design and implement an end-to-end **Machine Learning Operations (MLOps) platform** for early diabetes prediction using structured health data. The system supports the complete machine learning lifecycle, including data preprocessing, model training, deployment, monitoring, and continuous evaluation.

The platform enables real-time inference through a REST API, automated multi-model training with experiment tracking, model versioning via a centralized registry, and continuous monitoring for data drift. Apache Airflow is used to orchestrate training pipelines, while a React-based web dashboard provides user interaction for prediction, training, and drift analysis. Synthetic data generation is incorporated to simulate real-world traffic and support continuous monitoring.

The model is trained on a structured dataset containing demographic, clinical, vital sign, and medical history features, with diabetes prediction formulated as a **binary classification problem**. Class imbalance in the dataset is handled using SMOTE during training.

Evaluation Metrics

- **Accuracy:** Measures the overall correctness of the model's predictions (diabetic vs. non-diabetic).
- **Precision:** Evaluates how many of the predicted diabetic cases are actually true diabetic cases, which is important for reducing false-positive diagnoses.
- **Recall (Sensitivity):** Measures the model's ability to correctly identify actual diabetic patients (True Positives), which is critical for early detection and minimizing false negatives (missed cases).
- **F1-Score:** Provides a single metric that represents the harmonic mean of precision and recall, offering a balanced measure of performance, especially useful when dealing with the class imbalance in the diabetes dataset.
- **ROC-AUC Score:** Assesses the model's ability to distinguish between the diabetic and non-diabetic classes across all possible decision thresholds, indicating its overall discriminative ability.
- **Confusion Matrix:** Visualizes the breakdown of prediction results into True Positives, False Positives, True Negatives, and False Negatives, allowing for analysis of model error types, particularly the critical False Negatives.
- **Data Drift Detection Metrics:** Metrics like feature-wise mean shifts, overall drift scores, and maximum deviations are monitored using **Evidently AI** to detect changes in the distribution of incoming patient data over time. Drift alerts are generated when predefined thresholds are exceeded, indicating potential model performance degradation and the need for retraining.

1.3 Tools and Environmental Setup

The project is implemented using a modern, containerized technology stack to ensure scalability, reproducibility, and ease of deployment. **FastAPI** is used to develop the backend services for model inference, training, and monitoring, while **React** is used to build the frontend user interface. Machine learning models are developed using the **Scikit-learn** library.

Apache Airflow is employed for orchestrating and scheduling training pipelines, allowing automated and manual retraining workflows. **Comet ML** is used for experiment tracking, logging model metrics, hyperparameters, artifacts, and maintaining a centralized model registry. **Evidently AI** is integrated to perform data drift detection and monitoring.

All components are containerized using **Docker**, ensuring consistent execution across different environments. Synthetic data generators are used to simulate real-world inference traffic, supporting continuous monitoring and evaluation of the deployed model.

1.4 Literature survey :

1.4.1 Machine Learning Techniques for Diabetes Prediction

Early detection of diabetes using machine learning has been widely explored in recent years. Afolabi (2025) evaluated supervised learning algorithms such as Logistic Regression, k-Nearest Neighbors, and Random Forest and concluded that ensemble classifiers outperform linear models [1]. Rahman et al. (2025) demonstrated that advanced feature selection techniques significantly improve predictive accuracy by eliminating redundant clinical features [3].

Khokhar (2025) presented a systematic review of more than fifty studies and reported that Random Forest, Gradient Boosting, and neural networks consistently achieve higher performance than traditional statistical models [2]. Ahmed et al. (2024) proposed a demographic-specific prediction framework for women and showed that subgroup-based modeling improves generalization [4]. Abousaber et al. (2025) introduced a robust predictive framework using resampling and feature engineering techniques to enhance classification stability across datasets [5].

Recent studies emphasize interpretability and hybrid learning models. Hasan and Yasmin (2025) developed a hybrid CNN–LSTM model achieving high ROC-AUC scores [6]. Barman et al. (2025) applied explainable AI methods such as SHAP and identified glucose, BMI, and age as dominant predictors [9]. Rastogi et al. (2024) proposed an IoT-based ensemble learning framework for real-time diabetes monitoring [7], while Alzboon et al. (2025) confirmed that Random Forest and Neural Networks consistently outperform classical classifiers [8].

1.4.2 MLOps Tools and Deployment Frameworks

Although many studies report strong offline performance, few address production-level challenges. Hanchuk (2025) reported that ML systems fail in production due to lack of reproducibility and automation [11]. Reddy (2025) emphasized the need for CI/CD pipelines to automate training, validation, and deployment workflows [12].

John et al. (2025) showed that organizations adopting MLOps practices such as Docker-based containerization and experiment tracking achieve faster deployment cycles and improved traceability [13]. Singh et al. (2025) proposed scalable MLOps pipelines integrating Docker, orchestration tools, and automated retraining [14]. These works validate the use of Docker for environment consistency, Apache Airflow for workflow orchestration, and Comet ML for experiment tracking and model versioning.

1.4.3 Monitoring, Drift Detection, and Governance

Model degradation after deployment is a major concern in healthcare ML systems. Lee et al. (2024) proposed trustworthy ML frameworks integrating monitoring and drift detection [18]. Singh et al. (2025) demonstrated that automated retraining pipelines improve long-term stability [19].

Recent practical studies highlight the use of Evidently AI for detecting data drift by comparing real-time inference data with training distributions [20]. These monitoring systems enable early detection of performance degradation and support proactive retraining.

1.4.4 Research Gap and Motivation

Although extensive research exists on diabetes prediction and individual MLOps tools, very few studies integrate predictive modeling, containerized deployment, experiment tracking, drift detection, and automated retraining into a unified end-to-end framework. This motivates the proposed system, which combines Docker, Apache Airflow, Comet ML, and Evidently AI into a complete MLOps pipeline for early diabetes detection.

Chapter 2

2.1 Understanding MLOps, Risks, and Requirement Analysis

Machine Learning Operations (MLOps) refers to a set of practices that aim to automate, manage, and monitor the end-to-end lifecycle of machine learning models in production environments.

In healthcare applications such as diabetes prediction, MLOps becomes critical due to the sensitivity of medical data, the need for reliable predictions, and strict performance requirements.

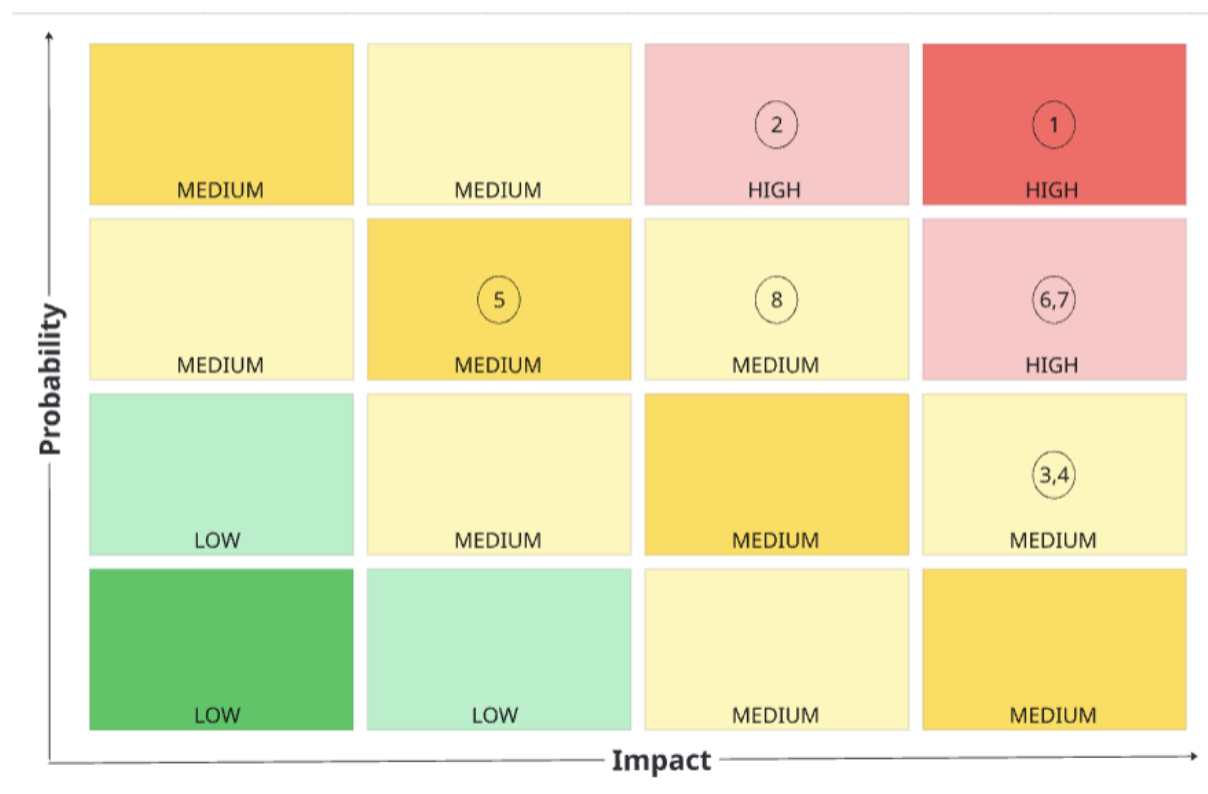
From a risk perspective, machine learning systems introduce new challenges beyond conventional software risks. These include data-related risks (such as bias and quality issues), model-related risks (such as drift and incorrect predictions), and system-level risks (such as API failures and security vulnerabilities).

The requirement analysis for this project focuses on:

- Reliable real-time inference with low latency
- Automated model training and deployment
- Continuous monitoring of data and predictions
- Version control and experiment tracking
- Risk mitigation through drift detection and logging

2.2 Risk Matrix

A **risk matrix** is used to evaluate and prioritize potential risks based on their **likelihood** of occurrence and **impact** on the system. Figure X illustrates the risk matrix for the proposed diabetes prediction MLOps system, where risks are categorized into low, medium, and high priority zones.



Identified Risks

1. **Data Bias** The dataset may not represent all population groups equally, leading to biased predictions for certain age groups, genders, or ethnicities.
2. **Data Quality Issues** Missing, incorrect, or inconsistent values in clinical features can negatively affect model training and inference accuracy.
3. **Incorrect Predictions** The model may produce false positives or false negatives, which is critical in a healthcare context where misclassification can impact patient outcomes.
4. **Security Risk** Medical data is sensitive, and unauthorized access or data leakage can lead to privacy violations and regulatory issues.
5. **Versioning Risk** Improper version control of datasets or models can result in loss of traceability and difficulty in reproducing results.
6. **Model Drift** Changes in the distribution of incoming patient data over time can degrade model performance if not detected and addressed.
7. **Deployment Failure** API or container failures during production deployment can disrupt real-time prediction services.
8. **Lack of Explainability** Complex models such as ensemble or deep learning models may be difficult to interpret, reducing trust among stakeholders.

Chapter 3

3.1 Dataset Description

This project utilizes a **Diabetes Prediction Dataset** obtained from **Mendeley Data**, a publicly available and peer-reviewed research data repository. The dataset is accessible at: <https://data.mendeley.com/datasets/7m7555vgrn/1>

The dataset consists of **5,437 patient records**, where each record represents an individual's demographic, physiological, and clinical information relevant to diabetes diagnosis. The dataset contains **15 attributes**, including age, gender, vital signs (pulse rate, blood pressure), metabolic indicators (glucose, BMI), anthropometric measurements (height and weight), and binary clinical history indicators such as hypertension, family history of diabetes,

cardiovascular disease, and stroke. The target variable represents the diabetic status of the individual and is defined as a binary classification label.

All features are provided in structured tabular format, making the dataset suitable for supervised machine learning–based classification tasks. The combination of metabolic, cardiovascular, and hereditary risk factors enables comprehensive modeling of diabetes risk. The dataset does not contain temporal or sequential dependencies and is intended for patient-level classification rather than longitudinal prediction.

3.2 Data Profiling, Cleaning and Preprocessing

A comprehensive data profiling process was carried out to assess the dataset’s structure, completeness, and statistical characteristics before model development.

3.2.1 Data Profiling

Initial profiling revealed that the dataset contains **no missing values**, indicating high data completeness. The dataset includes:

- **9 integer-type attributes**, primarily representing clinical indicators and vital signs
- **4 floating-point attributes**, representing continuous physiological measurements
- **2 categorical attributes**, namely gender and diabetic status

The dataset exhibits realistic clinical ranges for physiological parameters such as blood pressure, glucose levels, BMI, and pulse rate. Descriptive statistical analysis indicated sufficient variability across most features, particularly blood pressure, age, BMI, and weight, which are known to influence diabetes risk.

The target variable distribution reflects a medically realistic scenario, with diabetic cases forming a minority compared to non-diabetic cases, indicating a moderately imbalanced classification problem.

3.2.2 Data Cleaning

Although the dataset did not contain missing values, data cleaning focused on ensuring **consistency, validity, and usability** of the features. Categorical variables such as gender and diabetic status were encoded into numerical form to enable compatibility with machine learning algorithms.

Clinical attributes were examined for potential outliers and unrealistic values. The observed values fell within acceptable physiological ranges, and no records required removal. Binary clinical indicators (hypertension, family history, cardiovascular disease, and stroke) were validated to ensure consistency and correctness.

3.2.3 Data Preprocessing

Preprocessing steps were applied to prepare the dataset for feature extraction and model training:

- **Categorical Encoding:** Gender and diabetic status were converted into numerical encoded representations.
- **Feature Type Validation:** Data types were standardized to ensure numerical consistency across features.
- **Target Encoding:** The target variable was transformed into a binary encoded format suitable for supervised learning.
- **Data Integrity Checks:** Final validation ensured that all records were complete, correctly encoded, and aligned with model input requirements.

No normalization or scaling was applied at this stage, as scaling decisions were deferred to the model training phase based on algorithm requirements.

3.3 Feature Extraction

Feature extraction focuses on transforming raw attributes and deriving new variables that better represent the underlying physiological and clinical conditions related to diabetes. In this project, feature extraction was performed using **domain knowledge-driven transformations**, as the dataset is structured, tabular, and medically interpretable.

The original dataset includes physiological measurements such as glucose level, blood pressure, BMI, pulse rate, age, and binary clinical indicators. These features were retained as base predictors due to their established relevance in diabetes diagnosis.

To enhance representation, additional derived features were created:

- **Mean Arterial Pressure (MAP)** was computed from systolic and diastolic blood pressure values to represent overall arterial pressure load, which is more clinically meaningful than individual blood pressure readings.
- **Pulse Pressure** was derived as the difference between systolic and diastolic blood pressure to capture arterial stiffness, a known cardiovascular risk factor.
- **Glucose Category** was introduced by discretizing glucose values into clinically meaningful ranges, enabling the model to capture threshold-based metabolic risk.
- **Age Group** was derived by categorizing age into adult and senior groups to represent age-related diabetes risk stratification.

These extracted features improve the model's ability to capture complex physiological relationships while maintaining clinical interpretability, which is essential in healthcare-oriented machine learning systems.

3.4 Feature Selection

Feature selection was performed to identify the most informative attributes for diabetes prediction while reducing redundancy and improving generalization. Multiple feature selection techniques were applied to ensure robustness and consistency.

3.4.1 Correlation-Based Feature Selection

Correlation analysis was conducted to study relationships between input features and the target variable (diabetic). The results showed that **hypertension status** and **glucose level** have the strongest correlation with diabetes, followed by systolic and diastolic blood pressure. Age and weight demonstrated moderate correlation, while family history indicators and gender exhibited weak linear relationships.

Despite low correlation values for some features, clinically significant attributes were not removed solely based on correlation, as their influence may be non-linear or conditional.

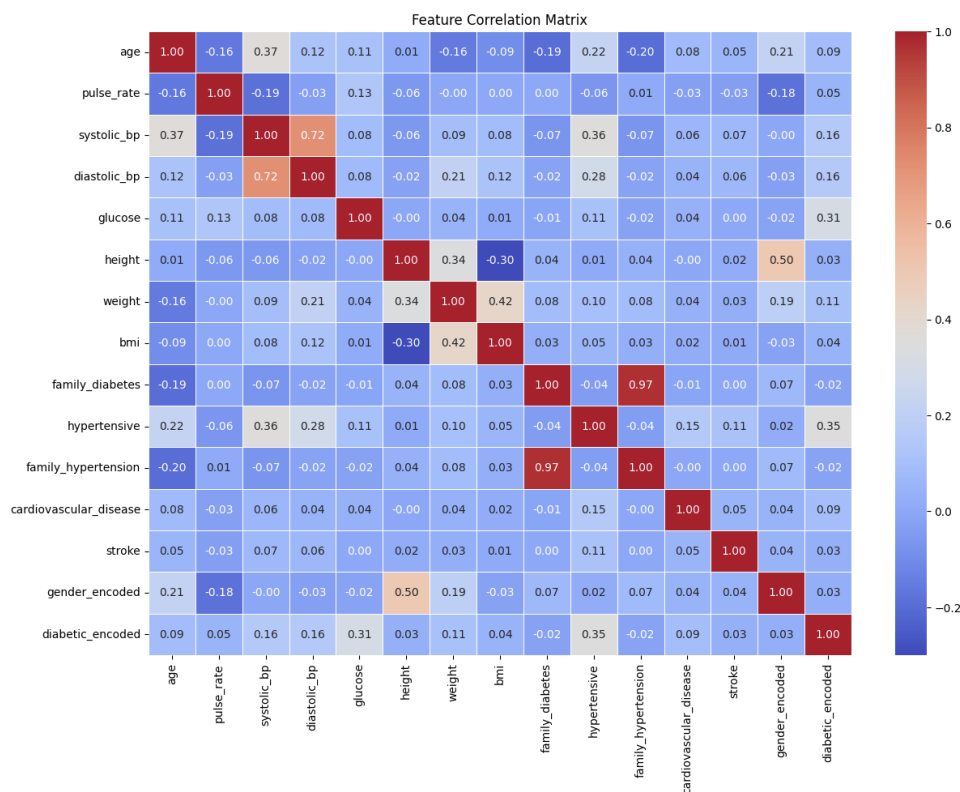


Figure 3.1 Correlation matrix of the features

3.4.2 Feature Importance Using Random Forest

A Random Forest classifier was trained to evaluate feature importance by measuring each feature's contribution to prediction accuracy. This method captures non-linear relationships and interactions between variables.

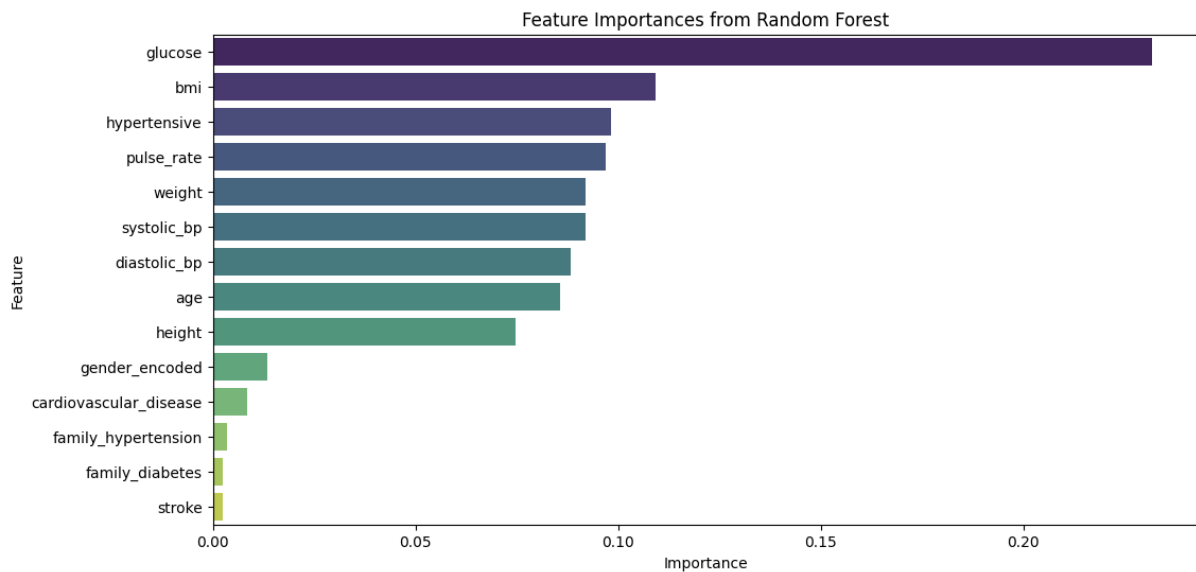


Figure 3.2 Features obtained from Random forest

The feature importance analysis revealed:

- **Glucose** as the most important predictor of diabetes.
- **BMI, hypertension status, pulse rate, weight, and blood pressure measurements** as strong contributors.
- **Age and height** as moderately important features.
- **Stroke, cardiovascular disease, and family history indicators** as having minimal contribution due to low occurrence and limited predictive power.

These findings align with established medical knowledge that metabolic and cardiovascular indicators are primary determinants of diabetes risk.

3.4.3 Statistical and Wrapper-Based Feature Selection Methods

To further validate feature relevance, multiple selection techniques were applied:

- **Variance Thresholding** showed that most physiological features exhibit sufficient variability, while stroke and cardiovascular disease display low variance.
- **Univariate Feature Selection (Chi-Square Test)** identified glucose, blood pressure, hypertension, and age as statistically significant predictors.
- **Recursive Feature Elimination (RFE)** selected glucose, diastolic blood pressure, weight, hypertension, and family diabetes as an optimal feature subset.
- **Sequential Feature Selection (SFS)** demonstrated that smaller subsets of clinically relevant features can achieve high predictive accuracy.

The consistency of key features across multiple methods confirms their reliability for diabetes prediction.

3.5 Model Design Approaches

Problem framing: Binary classification (diabetic), prioritize recall or precision depending on operational cost of false negatives vs false positives.

Model candidates: Baseline logistic regression (explainable), SVM (if margin-based separation helps), tree ensembles (RandomForest/XGBoost) for non-linear relationships.

Evaluation strategy: Use stratified cross-validation, track metrics (accuracy, precision, recall, F1, ROC-AUC) in Comet ML. Prefer F1 or recall for imbalanced positive class depending on product goals.

Ensembling & calibration: Consider stacked ensembles or voting if multiple models complement each other; apply probability calibration (Platt or isotonic) when decision thresholds matter.

Deployment-ready constraints: Latency budget, memory, interpretability, and reproducibility. Package model, preprocessing, and metadata (version, hyperparameters) as a single artifact.

Canary & rollback: Deploy new model as a canary, monitor metrics and drift, and support quick rollback to previous artifact in the model registry.

3.6 Governance and Data Version Control

Experiment and model tracking: Use Comet ML to record experiments, hyperparameters, metrics, artifacts, and registered model versions. Ensure each model has a unique experiment ID and associated metadata.

Data versioning: Version reference and training datasets using timestamped snapshots or dedicated data versioning tools. Track which model was trained on which data snapshot.

Reproducibility: Persist code snapshots, environment configuration, random seeds, and preprocessing pipelines. Log these details as part of the model artifact metadata.

Access and audit: Maintain an audit trail for training triggers, model approvals, and production deployments. Implement role-based access control where required.

Data privacy and compliance: Mask or anonymize sensitive attributes, apply data retention policies, and document data provenance. Ensure secure data transport and storage.

Monitoring and alerts: Define service-level objectives for model accuracy, latency, and drift thresholds. Generate alerts when metrics degrade or drift thresholds are exceeded. Log drift checks and expose drift reports.

Operational playbook: Document rollback procedures, retraining triggers, and ownership responsibilities. Automate retraining workflows while supporting human review for production model updates when required.

Chapter 4: Model Development and Evaluation

4.1 Model Building

Goal:

Select model families and design reproducible pipelines that meet performance, latency, and interpretability requirements.

Approach in this repo:

- **Baselines:** LogisticRegression for interpretability.
- **Non-linear models:** SVC (RBF) and RandomForestClassifier for higher accuracy.
- **Preprocessing:** StandardScaler, LabelEncoder saved with model artifacts (persist with joblib).
- **Pipeline pattern:** Combine preprocessing + sampling (SMOTE) + estimator in a scikit-learn Pipeline or use separate transformers + model object for clarity.
- **Production artifact:** Bundle model + preprocessing + metadata (version, training snapshot, random seed) and register to Comet ML.

4.2 Training, Validation and Testing

Data splits and strategy

- Use stratified train/test split to preserve class balance.

- For robust estimates, run stratified k-fold cross-validation (k=5 or 10); log fold metrics.
- Use a held-out test set for final evaluation; do not use test set for hyperparameter tuning.

Handling imbalance

- Apply SMOTE on training fold only.
- Evaluate with metrics robust to imbalance (precision, recall, F1, ROC-AUC).

Hyperparameter tuning

- GridSearchCV or randomized search inside cross-validation; log tried parameters and results to Comet.
- Avoid leaking validation information into final model; after tuning, retrain on full training set (or train + validation) and evaluate on holdout.

Reproducibility

- Fix random_state across splits and models.
- Persist training artifact: model, scaler, feature list, and Comet experiment ID.

Example commands

- Run training workflow through orchestration or invoke the training pipeline.
- In CI: run cross-validation unit tests to ensure training script reproduces metrics within tolerance.

4.3 Evaluation Metrics and Visualization

The performance of the machine learning models was evaluated using a combination of quantitative metrics and visual analysis to ensure both predictive accuracy and reliability under class imbalance conditions.

Evaluation Metrics

Accuracy was used as the primary metric for initial model comparison, as it provides an overall measure of correct predictions. However, since the dataset exhibits class imbalance, additional metrics were considered to better assess model effectiveness in identifying diabetic cases. These include **precision**, **recall**, and **F1 score**, which together capture the trade-off between false positives and false negatives. The **ROC-AUC** metric was used to evaluate the model's ability to distinguish between diabetic and non-diabetic classes across different decision thresholds.

Based on these metrics, the **Random Forest model** demonstrated superior overall performance, achieving an accuracy of **87.8%**, outperforming Logistic Regression (**83.6%**) and Support Vector Machine (**83.8%**). The Random Forest model also showed improved balance between recall and precision, making it suitable for deployment in a healthcare-oriented prediction system.

Visualization of Model Performance

Visual evaluation techniques were used to complement numerical metrics and provide deeper insights into model behavior.

A **confusion matrix** was generated for the selected model to analyze the distribution of true positives, true negatives, false positives, and false negatives. This visualization helped assess the model's effectiveness in correctly identifying diabetic cases while minimizing misclassification.

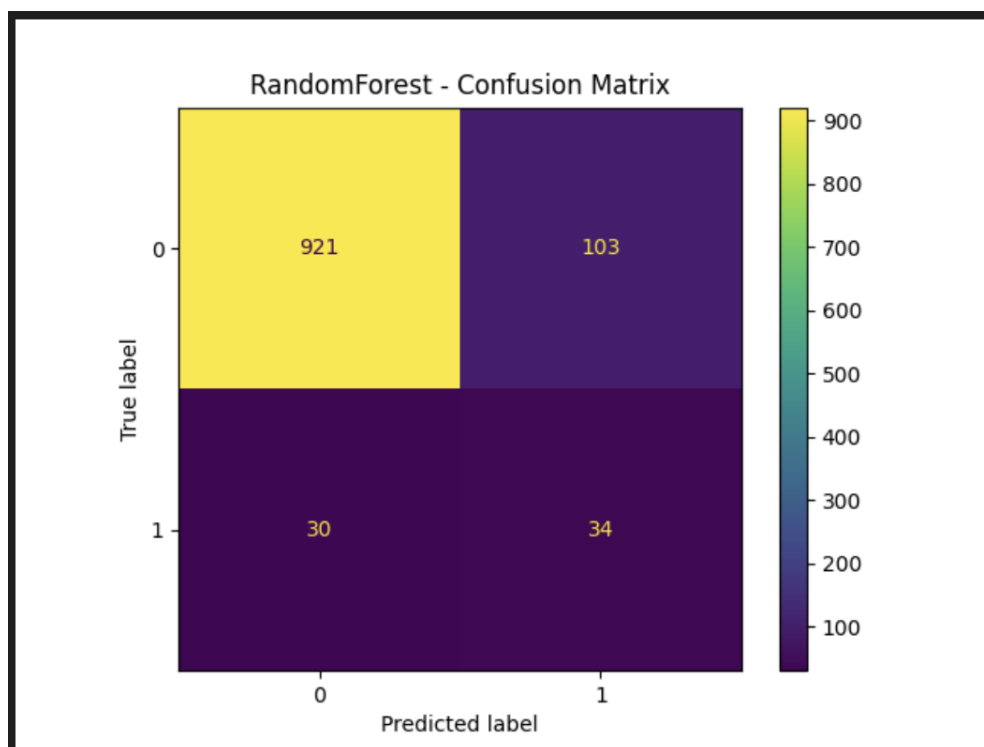


Figure 1 Confusion Matrix for the Random Forest Model

921 (True Negatives) → Non-diabetic patients correctly predicted as non-diabetic.

103 (False Positives) → Non-diabetic patients incorrectly predicted as diabetic.

30 (False Negatives) → Diabetic patients incorrectly predicted as non-diabetic (most critical error).

34 (True Positives) → Diabetic patients correctly identified.

The **Receiver Operating Characteristic (ROC) curve** was used to visualize the trade-off between true positive rate and false positive rate across varying thresholds. The area under the ROC curve provided an aggregate measure of classification performance independent of any single threshold.

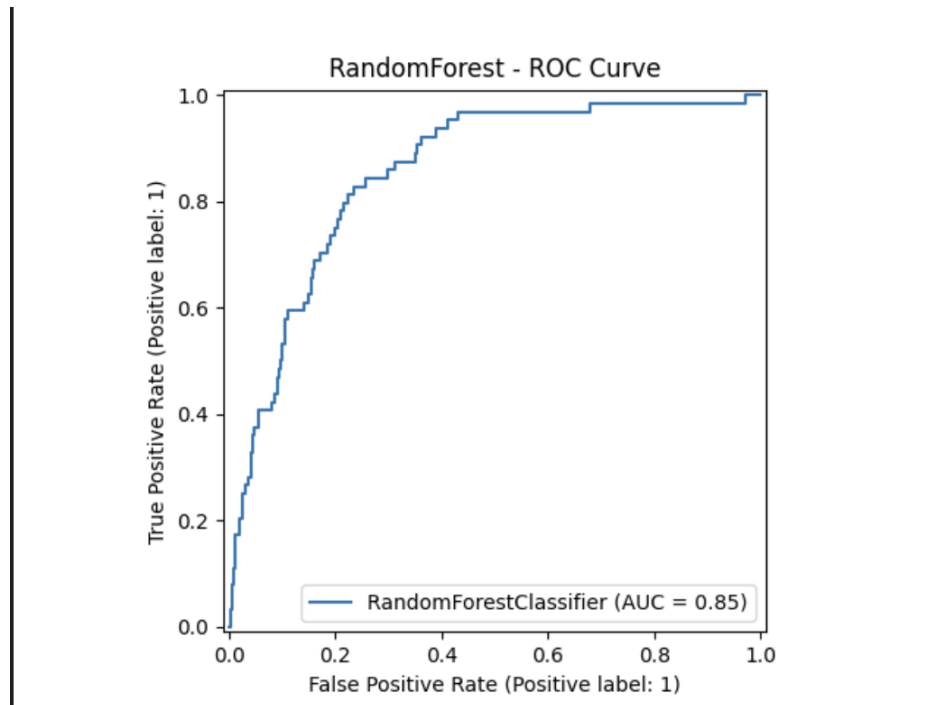


Figure 2 ROC Curve for the Random Forest Model

- There is an **85% chance** the model ranks a randomly chosen diabetic patient higher than a non-diabetic one
- This indicates **good discriminative ability**
- The model separates the two classes **significantly better than random guessing**

Curve shape insight

- Steep rise at low false positive rates
 - The model detects many true diabetics early
- Performance is stable across thresholds

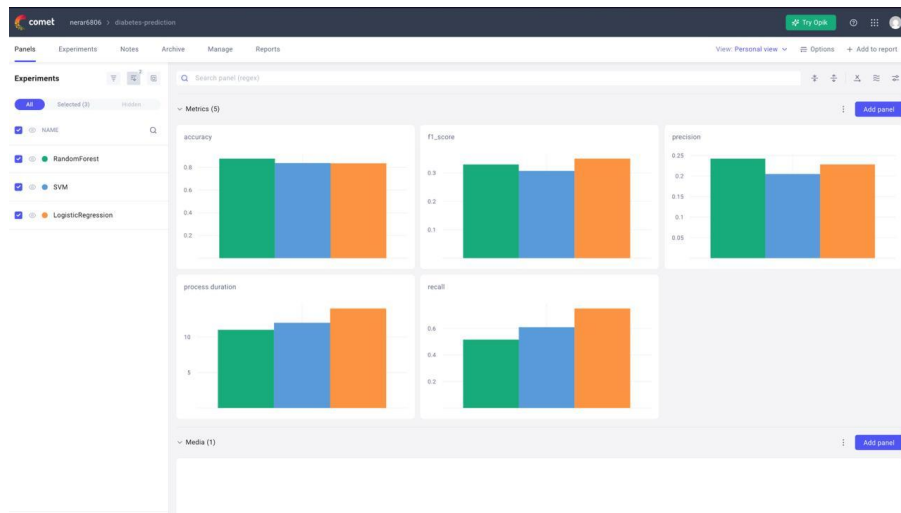


Figure 4.2 AUC, Recall, F1-score of all models on Comet ML

4.4 Experiment Tracking

Tooling:

Comet ML.

What to log

- Hyperparameters
- Training, validation, and test metrics
- Model file artifacts
- Plots (confusion matrix, ROC)
- Code snapshot
- Environment (requirements)
- Model registration name and version or tag

Workflow

- For each experiment, create a Comet Experiment, set a descriptive name, and log parameters, metrics, and artifacts.
- After selecting a production candidate, register the model and note the production tag in metadata.
- Use the Comet dashboard to compare runs, inspect metrics, and download artifacts programmatically via API.

Reproducibility

- Store experiment IDs and mapping to data snapshot in a training log.

NAME	TAGS	SERVER END TIME	FILE NAME	DURATION	USER NAME	PROCESS DURATION	F1_SCORE	ACCURACY	PRECISION	RECALL	CLASS
RandomForest		12/26/25 10:46 AM	train_and_log.py	00:00:11	nerar6806	11	0.33	0.876838	0.242647	0.515625	
SVM		12/26/25 10:45 AM	train_and_log.py	00:00:09	nerar6806	12	0.307087	0.838235	0.205263	0.609375	balance
LogisticRegression		12/26/25 10:45 AM	train_and_log.py	00:00:11	nerar6806	14	0.350365	0.836397	0.228571	0.75	balance

Figure 4.4 Experiment tracking of Models in Comet ML

4.5 Interpreting Model Results and Error Analysis

Model interpretability

- For linear models: coefficients and odds ratios.
- For tree ensembles: feature importances and partial dependence plots.
- Model-agnostic: SHAP or LIME for local explanations; log typical SHAP summaries for production incidents.

Error analysis workflow

- Collect misclassified examples from test set and from production predictions with any available labeled feedback.
- Group errors by feature buckets (age ranges, high glucose, etc.) and by model confidence (low probability predictions).
- Identify root causes: data quality issues (missing or mis-typed features like gender), feature drift, label noise, or insufficient representation.

Actionable outputs

- If errors cluster by subgroup → consider feature interactions, data augmentation, or separate models per subgroup.
- If calibration is poor → apply probability calibration (isotonic or Platt).
- Document corrective steps in a retraining ticket including dataset snapshot, hyperparameter change, and expected metric improvement.

Operationalizing analysis

- Automate periodic error sampling and produce a short error report attached to experiment runs.
- Maintain an incident log linking production failures to retraining events.

Phase 2: Deployment, Monitoring and Governance

Chapter 5: Model Analysis and Refinement:

This chapter focuses on post-development improvements to the machine learning model after initial training and evaluation. It includes performance reassessment, tuning strategies, interpretability techniques, model versioning, and systematic documentation of changes. These activities ensure that the deployed model remains robust, traceable, and suitable for real-world usage within an MLOps framework.

5.1 Revisiting Model Performance :

After deploying the initial version of the diabetes prediction model, its performance is revisited using both offline validation metrics and live inference data collected during serving. The goal of this step is to verify whether the model maintains consistent performance when exposed to real-world data distributions.

Performance indicators such as accuracy, precision, recall, F1-score, and ROC-AUC are compared between the training/validation phase and post-deployment inference logs. Prediction outputs stored in `recent_predictions.csv` provide an audit trail that enables evaluation of real-time behavior. Any observable degradation in metrics may indicate data drift, concept drift, or overfitting during training.

This reassessment ensures that the model continues to meet the originally defined evaluation criteria and business objectives before further refinement steps are applied.

5.2 Hyperparameter Tuning and Performance Optimization :

To improve predictive performance and generalization, hyperparameter tuning is performed on the selected machine learning models implemented using scikit-learn. Parameters such as regularization strength, class weights, solver configurations, and sampling strategies are adjusted to address class imbalance and variance issues.

The tuning process is executed through repeated training runs orchestrated by the training pipeline and logged using the experiment tracking framework. Each run records model parameters, evaluation metrics, and artifacts, enabling systematic comparison of results. Oversampling techniques from imbalanced-learn are incorporated where required to mitigate skewed class distributions in the diabetes dataset.

By iteratively refining hyperparameters and validating results across multiple runs, the final model configuration achieves improved stability and predictive reliability

5.3 Interpretability :

Model interpretability is a critical requirement in healthcare-related machine learning applications. To ensure transparency, the trained model's predictions are analyzed to understand feature influence and decision behavior.

Feature importance scores and coefficient analysis (for linear models) are examined to identify which clinical and demographic attributes most strongly influence the diabetes prediction outcome. Error analysis is performed on misclassified samples to determine whether incorrect predictions arise from noisy inputs, insufficient feature representation, or model limitations.

Interpretability results help establish trust in the model's decisions and support compliance with governance and ethical AI principles, especially in sensitive domains such as healthcare.

5.4 Model Versioning and Registry :

Model versioning is implemented to ensure reproducibility, traceability, and controlled deployment of trained models. Each successful training run produces a versioned model artifact that is stored either locally in the `models` directory or published to a centralized registry.

The training orchestration script manages the saving and registration of model artifacts along with associated metadata, including training parameters, dataset version, and evaluation metrics. The model loading logic retrieves the appropriate version during inference, ensuring consistency between training and serving environments.

This structured versioning approach enables rollback to previous models if issues are detected and supports parallel experimentation without disrupting production systems.

5.5 Documentation of Model Updates :

All modifications made to the model during refinement—including hyperparameter changes, feature updates, retraining cycles, and performance improvements—are documented systematically. Each model version is accompanied by descriptive metadata outlining the rationale for changes, observed improvements, and any known limitations.

Documentation artifacts include experiment logs, evaluation summaries, and version notes maintained alongside model files and registry entries. This practice ensures auditability, supports team collaboration, and facilitates future maintenance or handover of the system.

Comprehensive documentation also plays a crucial role in regulatory compliance and long-term governance of the deployed machine learning solution.

Chapter 6: Implementing CI/CD Pipeline

This chapter describes the implementation of a Continuous Integration and Continuous Deployment (CI/CD) pipeline tailored for machine learning workflows. Unlike traditional software systems, ML pipelines must handle data dependencies, model artifacts, and experiment tracking in addition to application code. The CI/CD setup in this project automates training, validation, testing, and deployment processes to ensure reliability and reproducibility.

6.1 Overview of CI/CD for Machine Learning :

CI/CD for machine learning extends conventional DevOps practices by incorporating data validation, model training, evaluation, and monitoring into the pipeline. Continuous Integration focuses on automatically validating code changes, retraining models when required, and ensuring that experiments remain reproducible.

In this project, CI/CD enables seamless transitions from model development to deployment by integrating orchestration tools, containerization, and automated testing. This approach reduces manual intervention, minimizes errors, and ensures faster iteration cycles for model improvements.

6.2 Tools Setup :

The CI/CD workflow is managed by a dedicated automation tool that integrates with the core MLOps components. This tool is responsible for :

- **Triggering:** Kicking off the workflow automatically upon code or data changes.
- **Containerization:** Running **Docker** builds to create the standardized images for the backend service (`app.py`), the monitoring service (`monitor.py`), and the frontend dashboard.
- **Orchestration:** Triggering the scheduled training pipeline managed by **Apache Airflow** (`diabetes_train_dag.py`) via an API call, ensuring that all components (data preparation, training, evaluation, and logging to **Comet ML**) run in the correct sequence.
- **Deployment:** Using **Docker Compose** to manage and deploy the multi-service application (API, monitor, frontend) on the target infrastructure, guaranteeing version consistency.

6.3 Automating Model Training and Testing :

Model training and testing are automated as part of the CI workflow. When triggered, the pipeline executes the training orchestration script, which performs data preprocessing, model training, evaluation, and experiment logging.

Unit tests and integration tests verify critical components such as model loading, inference logic, and registry interactions. Test scripts validate that trained models can be successfully

retrieved from the registry and used for prediction without errors. This automated verification step prevents faulty models or incompatible artifacts from reaching production.

6.4 Building and Running a Deployment Pipeline :

Once a model passes training and validation stages, the deployment pipeline builds Docker images for the backend inference service, monitoring components, and frontend dashboard. These images are then deployed using Docker Compose, enabling coordinated startup of all services.

The backend service exposes inference and health-check endpoints, allowing downstream services and the frontend to interact with the deployed model. Deployment pipelines ensure that the correct model version is loaded during service startup, maintaining alignment between training outputs and serving behavior.

This automated deployment mechanism enables rapid updates while maintaining service availability and system stability.

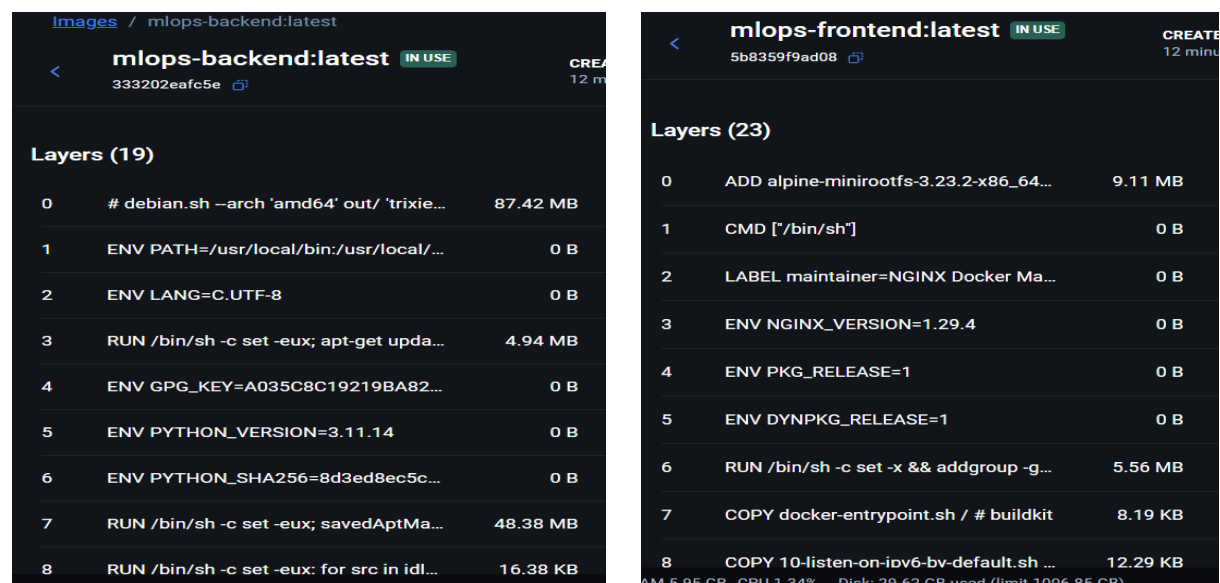
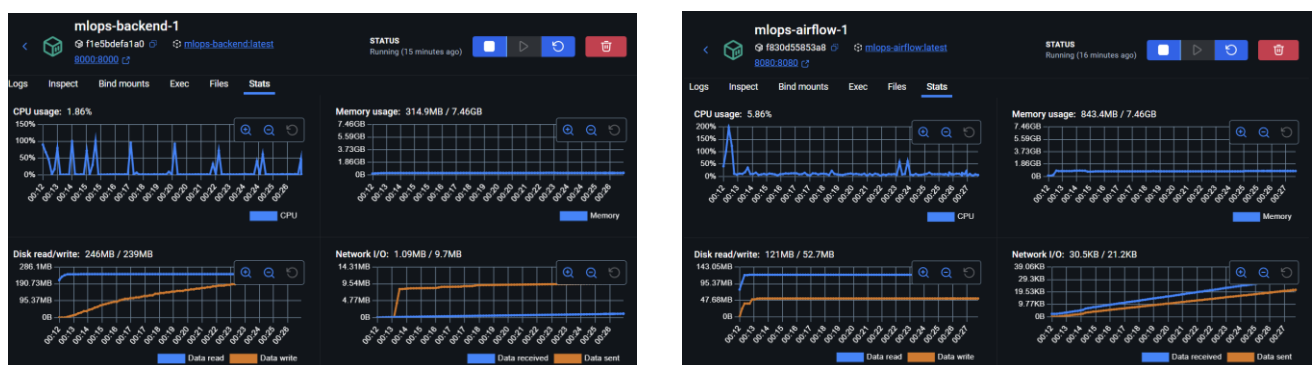


Figure 6.1 Layers of the Docker Image build



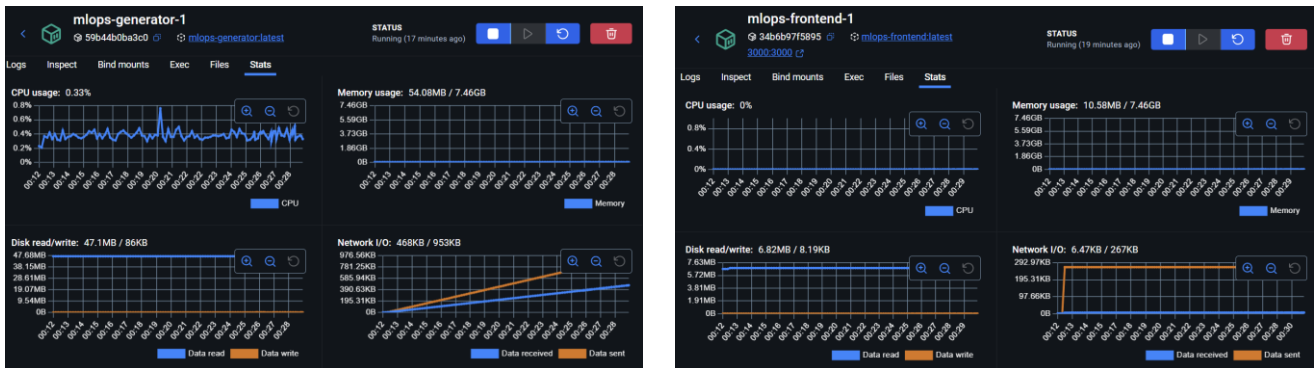


Figure 6.2 Performance Stats of the Docker Image

Chapter 7: Monitoring and Tracking Model Lifecycle

This chapter focuses on continuous monitoring of the deployed machine learning model to ensure sustained performance, reliability, and fairness over time. Since real-world data distributions may change after deployment, proactive monitoring and lifecycle management are essential components of an effective MLOps system. This project integrates automated monitoring, drift detection, logging, and retraining mechanisms to maintain model quality in production.

7.1 Model Monitoring Concepts and KPIs :

Model monitoring involves tracking both system-level and model-level indicators to detect performance degradation or abnormal behavior. Key Performance Indicators (KPIs) monitored in this project include prediction accuracy trends, class distribution changes, data quality metrics, and inference consistency.

Operational metrics such as service availability, response time, and error rates are also observed to ensure the reliability of the deployed inference API. Prediction logs collected during runtime provide the foundation for calculating these KPIs and enable retrospective analysis of model behavior.

Monitoring these indicators allows early identification of issues such as data drift, concept drift, or infrastructure-related failures.

7.2 Tools: EvidentlyAI and Frontend Visualization :

A combination of monitoring and visualization tools is used to track the health and performance of the ML system. EvidentlyAI is employed to analyze data drift and prediction drift by

comparing live inference data collected during serving with a reference dataset generated during training. The drift analysis results are compiled into a detailed HTML report (produced by the `drift.py` script) for interpretability. The React frontend, utilizing the Recharts library, provides a dashboard to display these prediction trends and drift reports, giving users a clear, accessible view of the model's performance in production.

7.3 Detecting Concept Drift and Data Drift :

Data drift occurs when the statistical properties of input features change over time, while concept drift refers to changes in the relationship between input features and the target variable. Both forms of drift can significantly impact model accuracy if left unaddressed.

In this project, drift detection is performed by comparing recent prediction data collected during inference with a reference dataset derived from the training phase. Statistical tests and distribution comparisons are applied to identify significant deviations. The resulting drift reports provide both quantitative metrics and visual summaries, enabling clear interpretation of detected changes.

Automated drift detection ensures that model degradation is identified early and corrective actions can be taken promptly.

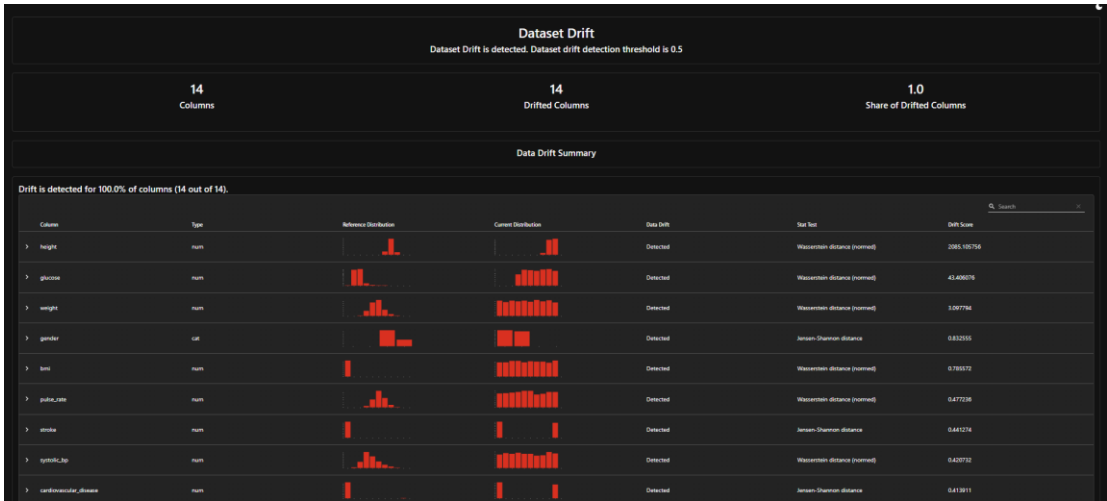


Figure 7.1 Evidently AI Drift detection

7.4 Model Retraining and Redeployment :

When monitoring systems detect significant drift or performance decline, the model retraining process is triggered. Retraining may involve updating the training dataset with newer data, recalibrating preprocessing steps, and optimizing model parameters.

The retraining workflow follows the same automated pipeline used during initial development, ensuring consistency and reproducibility. Once retrained, the updated model undergoes validation before being registered as a new version and deployed through the CI/CD pipeline.

This closed-loop process enables continuous learning and adaptation of the model to evolving data patterns.

7.5 Logging, Alerting, and Dashboard Setup :

Prediction logging plays a crucial role in monitoring and auditing model behavior. Each inference request and response is logged along with relevant metadata such as timestamps and feature values. These logs support performance analysis, drift detection, and compliance requirements.

Alerting mechanisms can be configured to notify stakeholders when predefined thresholds are exceeded, such as sudden spikes in drift metrics or service failures.

Together, logging, alerting, and dashboards provide transparency and operational control over the deployed machine learning system.

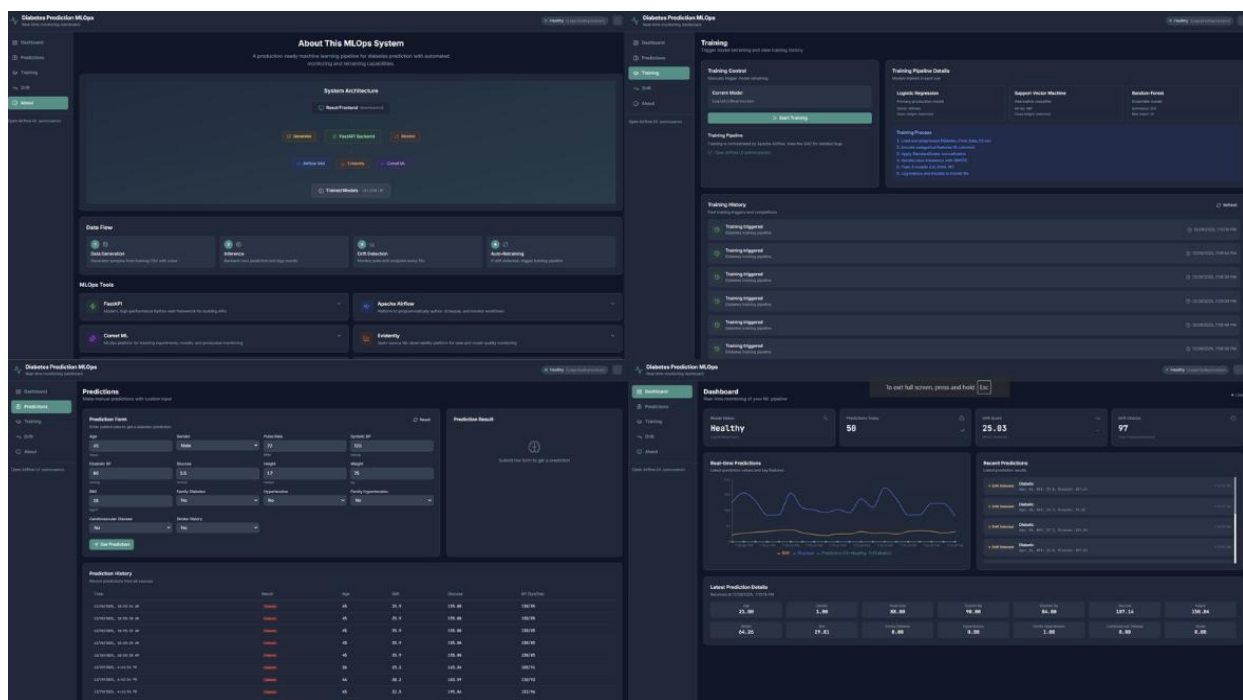


Figure 7.2 Dashboard UI for the Diabetes Risk Prediction

Chapter 8: Performance Evaluation and Governance

This chapter addresses post-deployment evaluation, governance mechanisms, and ethical considerations associated with deploying machine learning models in production. Continuous evaluation and structured governance ensure that the system remains accurate, fair, transparent, and compliant with best practices throughout its lifecycle.

8.1 Post-Deployment Performance Review

After deployment, the performance of the diabetes prediction model is periodically reviewed using real-world inference data. Prediction logs collected during serving are analyzed to assess whether the model continues to meet the predefined evaluation metrics established during the development phase.

Comparisons are made between training, validation, and post-deployment performance to identify discrepancies caused by data drift, changing population characteristics, or operational constraints. This review process ensures that any degradation in model effectiveness is detected early and addressed systematically.

8.2 Continuous Improvement via Feedback Loops :

A continuous feedback loop is established to improve model performance over time. Monitoring outputs, drift reports, and user feedback are integrated into the retraining pipeline, allowing the system to adapt to new data patterns and evolving requirements.

This iterative improvement process enables informed decision-making regarding when to retrain, tune, or replace models. Feedback loops also support proactive maintenance rather than reactive fixes, which is a key objective of MLOps-driven systems.

8.3 Governance and Ethical AI Practices :

Governance frameworks are implemented to ensure responsible and ethical use of machine learning within the system. Given the healthcare context of diabetes prediction, special emphasis is placed on transparency, fairness, and accountability.

Ethical AI practices include monitoring for bias, validating model decisions, and ensuring that predictions are used as decision-support tools rather than definitive diagnoses. Access controls, version tracking, and audit logs further strengthen governance by enabling traceability and compliance with institutional or regulatory requirements.

8.4 Documentation: Model Cards and Data Cards :

Comprehensive documentation is maintained to support governance and transparency. Model cards describe the purpose, architecture, training data, evaluation metrics, limitations, and intended use cases of each model version. Data cards document dataset sources, preprocessing steps, data quality considerations, and licensing information.

These artifacts provide a standardized way to communicate model and data characteristics to stakeholders and support auditability, reproducibility, and long-term maintenance of the system.

8.5 Audit and Review Checklist :

An audit and review checklist is used to periodically assess the readiness and compliance of the deployed machine learning system. The checklist includes verification of model performance, data integrity, versioning practices, documentation completeness, and monitoring effectiveness.

Regular audits ensure that the system adheres to organizational policies and MLOps best practices. This structured review process helps identify risks, improve operational discipline, and maintain confidence in the deployed solution.

Appendix

A.1 List of MLOps Tools, Features, Pros and Cons :

This project integrates a diverse set of MLOps tools to manage the end-to-end machine learning lifecycle for diabetes prediction.

Tool	Core Feature in Project	Pros	Cons
FastAPI	Backend API for inference (<code>/predict</code>), training trigger (<code>/train</code>), and monitoring (<code>/drift</code>).	High performance, automatic interactive API documentation (Swagger/OpenAPI), easy to containerize.	Requires manual handling of asynchronous tasks (e.g., triggering Airflow DAGs).
Apache Airflow	Orchestration and scheduling of the training pipeline (<code>diabetes_train_dag.py</code>).	Highly scalable for complex dependency management, robust scheduling capabilities.	Requires dedicated setup/infrastructure (e.g., scheduler, webserver, workers).
Comet ML	Experiment Tracking and Model Registry. Logs metrics, artifacts, and manages model versions.	Centralized logging of experiments, easy comparison of runs, programmatic model retrieval.	External dependency, potential cost implications for large-scale use.
Evidently AI	Data and Prediction Drift Detection. Generates drift reports (<code>drift.py</code>).	Specialized tool for ML monitoring, produces visual and numerical reports.	Needs a dedicated monitoring service (<code>monitor.py</code>) to automate checks and retraining.

Docker / Docker Compose	Containerization of all components (backend, frontend, Airflow, monitor, generator).	Ensures environment consistency and reproducibility across development and deployment.	Initial overhead in writing and maintaining Dockerfiles and docker-compose.yml .
React	Frontend User Interface. Displays prediction stream, drift reports, and training logs.	Modern, component-based architecture for building interactive dashboards.	Requires expertise in JavaScript/TypeScript and a separate build/deployment process.

A.2 Tool Installation Guides :

Detailed, step-by-step installation guides for the individual tools are outside the scope of this MLOps Report but are critical for environment setup. For a complete deployment, the installation process would involve:

1. Installing **Python 3** and core ML libraries (**scikit-learn**, **pandas**) via [pip](#).
2. Installing **Docker** and **Docker Compose** for local and production deployment.
3. Setting up the **Apache Airflow** environment, often via a community Docker image or Helm charts.
4. Installing **Node.js** and **npm/yarn** to manage the React frontend dependencies.
5. Configuring **Comet ML** API keys for experiment tracking and model registry access.

A.3 Dataset Referencing and Licensing

Dataset: Structured clinical and demographic health parameters.

Format: CSV format.

Target Variable: Binary classification (Diabetic/Non-Diabetic).

Key Characteristics:

- Includes features like age, gender, vital signs, clinical measurements (e.g., glucose, blood pressure, BMI), and medical history indicators.
- The dataset exhibits **class imbalance**, which is addressed during training using the **Synthetic Minority Over-sampling Technique (SMOTE)** from the [imbalanced-learn](#) library.
- **Data Profiling, Cleaning, and Preprocessing** (Section 3.2) were performed to handle missing values, validate ranges, and apply feature scaling.

A.4 Folder Structure :

The project follows a modular structure organized by component responsibility, supporting containerization and clear separation of concerns.

MLOPS/

```
|— airflow/
|   |— Dockerfile
|   └— requirements.txt
|— backend/
|   |— app.py
|   |— create_reference_data.py
|   |— Dockerfile
|   |— drift.py
|   |— drift_report.html
|   |— inference.py
|   |— load_from_registry.py
|   |— prediction_logger.py
|   |— recent_predictions.csv
|   |— requirements.txt
|   |— test_registry_load.py
|   |— train_and_log.py
|   |— train_reference.csv
|   |— training.py
|   |— models/
|   └— pycache/
|— dags/
```



```
| |— diabetes_train_dag.py
| |— pycache/
|— frontend/
| |— .env
| |— .gitignore
| |— Dockerfile
| |— eslint.config.js
| |— index.html
| |— nginx.conf
| |— package.json
| |— postcss.config.js
| |— README.md
| |— tsconfig.app.json
| |— tsconfig.json
| |— tsconfig.node.json
| |— vite.config.ts
| |— public/
| |— src/
| | |— App.tsx
| | |— index.css
| | |— main.tsx
| | |— assets/
| | |— components/
| | |— hooks/
| | |— pages/
```

```

|   |— services/
|   |— types/
|— generator/
|   |— Dockerfile
|   |— generator.py
|— logs/
|   |— dag_id=diabetes_training_pipeline/
|   |   |— ... (individual run folders)
|   |— dag_processor_manager/
|   |— scheduler/
|   |   |— latest
|— monitor/
|   |— Dockerfile
|   |— monitor.py
|— .gitignore
|— airflow-webserver.pid
|— airflow.cfg
|— dev.sh
|— Diabetes_Final_Data_V2.csv
|— docker-compose.yml
|— run.sh
|— standalone_admin_password.txt
|— webserver_config.py

```

MLOPS/

- └─ backend/
 - | └─ app.py # FastAPI app (API endpoints)
 - | └─ train_and_log.py # Training script, logs to Comet ML
 - | └─ load_from_registry.py # Loads model from Comet/local
 - | └─ drift.py # Evidently drift checks and report generation
 - | └─ training.py # Client to trigger Airflow DAG via REST API
 - | └─ monitor.py # Drift monitor, triggers retraining on drift
 - | └─ requirements.txt # Python dependencies (incl. pandas, scikit-learn, imbalanced-learn)
 - | └─ models/ # Local fallback for model artifacts
- └─ dags/
 - | └─ diabetes_train_dag.py # Apache Airflow DAG definition
- └─ generator/
 - | └─ generator.py # Synthetic data generator
- └─ frontend/
 - | └─ src/ # React/TypeScript source code
 - | └─ package.json # Frontend dependencies (incl. React, Recharts, Axios)
- └─ docker-compose.yml # Orchestrates all services (backend, frontend, airflow, etc.)
- └─ README.md

A.5 References

The following list contains the literature and external sources consulted for the report.

- [1] S. Afolabi, “Predicting diabetes using supervised machine learning,” *Machine Learning with Applications*, 2025.
<https://www.sciencedirect.com/science/article/pii/S2949953425000013>
- [2] P. B. Khokhar, “Advances in artificial intelligence for diabetes prediction: A systematic review,” *Artificial Intelligence in Medicine*, 2025.
<https://www.sciencedirect.com/science/article/pii/S09333365725000673>
- [3] F. Rahman et al., “Diabetes prediction using feature selection algorithms,” *Diagnostics*, 2025.
<https://www.mdpi.com/2075-4418/15/20/2622>
- [4] A. Ahmed et al., “Machine learning algorithm-based prediction of diabetes,” *Healthcare*, 2024.
<https://www.mdpi.com/2227-9032/13/1/37>
- [5] A. Abousaber et al., “A robust predictive framework for diabetes diagnosis,” *Frontiers in Artificial Intelligence*, 2025.
<https://www.frontiersin.org/articles/10.3389/frai.2024.1499530/full>
- [6] M. Hasan and F. Yasmin, “Hybrid CNN-LSTM for diabetes prediction,” *arXiv*, 2025.
<https://arxiv.org/abs/2505.07036>
- [7] R. Rastogi et al., “IoT-based ensemble ML framework for diabetes prediction,” *ETASR*, 2024.
<https://etasr.com/index.php/ETASR/article/view/8869>
- [8] S. Alzboon et al., “Comparative study of ML techniques for early diabetes prediction,” *arXiv*, 2025.
<https://arxiv.org/abs/2506.10180>
- [9] P. B. Khokhar et al., “Explainable AI for diabetes prediction,” *arXiv*, 2025.
<https://arxiv.org/abs/2501.18071>
- [10] A. Sefid et al., “Predicting diabetes risk using ML – YaHS study,” *IJDO*, 2025.
<https://publish.kne-publishing.com/index.php/IJDO/article/view/19267>
- [11] D. Hanchuk, “Implementing MLOps practices,” *CEUR Workshop Proceedings*, 2025.
<https://ceur-ws.org/Vol-3918/paper404.pdf>
- [12] V. Reddy, “MLOps and continuous ML delivery pipelines,” *ARJCSIT*, 2025.
<https://arjonline.org/papers/arjcsit/v8-i1/7.pdf>

- [13] M. John et al., “An empirical guide to MLOps adoption,” *Information and Software Technology*, 2025.
<https://www.sciencedirect.com/science/article/pii/S0950584925000643>
- [14] R. Singh et al., “Building scalable MLOps pipelines,” *ResearchGate*, 2025.
<https://www.researchgate.net/publication/398939158>
- [15] B. Eken et al., “Multivocal review of MLOps practices,” *ACM Computing Surveys*, 2025.
<https://dl.acm.org/doi/full/10.1145/3747346>
- [16] D. Kreuzberger et al., “MLOps: Overview, definition and architecture,” *arXiv*, 2022.
<https://arxiv.org/abs/2205.02302>
- [17] R. Mehta, “MLOps: Changes and tools,” *IJERT*, 2025.
<https://www.ijert.org/mlops-changes-and-tools-adapting-machine-learning-workflows-for-enhanced-efficiency-and-scalability>
- [18] J. Lee et al., “Towards trustworthy ML in production,” *arXiv*, 2024.
<https://arxiv.org/abs/2410.21346>
- [19] P. Singh et al., “ML DevOps adoption in practice,” *arXiv*, 2025.
<https://arxiv.org/abs/2502.05634>
- [20] Evidently AI, “Practical ML monitoring & drift detection,” 2024.
<https://www.evidentlyai.com/blog/mlops-zoomcamp-monitoring>

A6. Further Reading :

1. **FastAPI** – [FastAPI Documentation](#)
2. **React** – [React Documentation](#)
3. **Apache Airflow** – [Apache Airflow Documentation](#)
4. **Comet ML** – [Comet Documentation](#)
5. **Evidently AI** – [Evidently AI Documentation](#)
6. **Docker** – [Docker Documentation](#)

Dataset Link : <https://data.mendeley.com/datasets/7m7555vgrn/1>